



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización y Arquitectura de Computadoras

Reporte - Construcción de Máquinas de Estados Usando Memorias

INTEGRANTES EQUIPO 8:

- Saldaña Navarrete Andrea - *Grupo Teoria: 1*
- Tepal Briseño Hansel Yael - *Grupo Teoria: 1*
- Ugartechea González Luis Antonio - *Grupo Teoria: 1*

NOMBRE DEL PROFESOR: Ing. Diana Karen Pérez Pérez

GRUPO: 06

SEMESTRE: 2027-1

FECHA DE ENTREGA: 1 de octubre del 2026

Objetivo

Familiarizar al alumno con la construcción de máquinas de estados mediante el uso de memorias, aplicando el método de direccionamiento por trayectoria.

Introducción

En el ámbito del diseño digital, el método de direccionamiento por trayectoria permite implementar cartas ASM almacenando el comportamiento lógico directamente en una memoria. Este tipo de direccionamiento guarda el estado siguiente y las salidas de cada estado en una localidad específica [1]. La porción de la memoria que se encarga de indicar cuál será el estado siguiente recibe el nombre de "liga", mientras que otra porción se destina a la parte de las salidas [1]. Para que el sistema determine la dirección de memoria exacta a la que debe acceder en cada pulso de reloj, se concatenan los bits de la liga del estado presente junto con las variables de entrada [1], [2].

Para poder construir el contenido de la tabla de memoria, es un requisito indispensable asignar previamente a cada estado de la carta ASM una codificación o representación binaria [1]. La cantidad mínima de bits requeridos depende del número total de estados de la máquina; por ejemplo, si se cuenta con 5 estados, se requieren por lo menos 3 bits para representarlos de manera binaria (ya que $2^3 = 8$ combinaciones posibles) [2]. Asimismo, la regla fundamental al momento de estructurar el contenido de memoria establece que, para cada estado, es necesario considerar todas las posibles combinaciones de las variables de entrada, aun cuando algunas de ellas no se utilicen de manera explícita en ese estado determinado [1].

Finalmente, el dimensionamiento y la organización de la memoria dependen directamente de la suma de variables. Si se diseña una carta ASM de 5 estados (3 bits de liga), 3 entradas y 4 salidas, la memoria requerirá un bus de direcciones de 6 bits (3 de liga + 3 de entrada) y palabras de 7 bits de longitud (3 de liga siguiente + 4 de salida). Esto da como resultado una organización de memoria de $2^6 \times 7$, es decir, de 64×7 bits [2].

Desarrollo

1. Diseñe una carta ASM con las siguientes características:

- 5 estados
- 3 entradas
- 4 salidas

- 2 con lógica estándar
- 2 con lógica negada
- Dada una entrada, colocar salidas condicionales en ambos caminos.

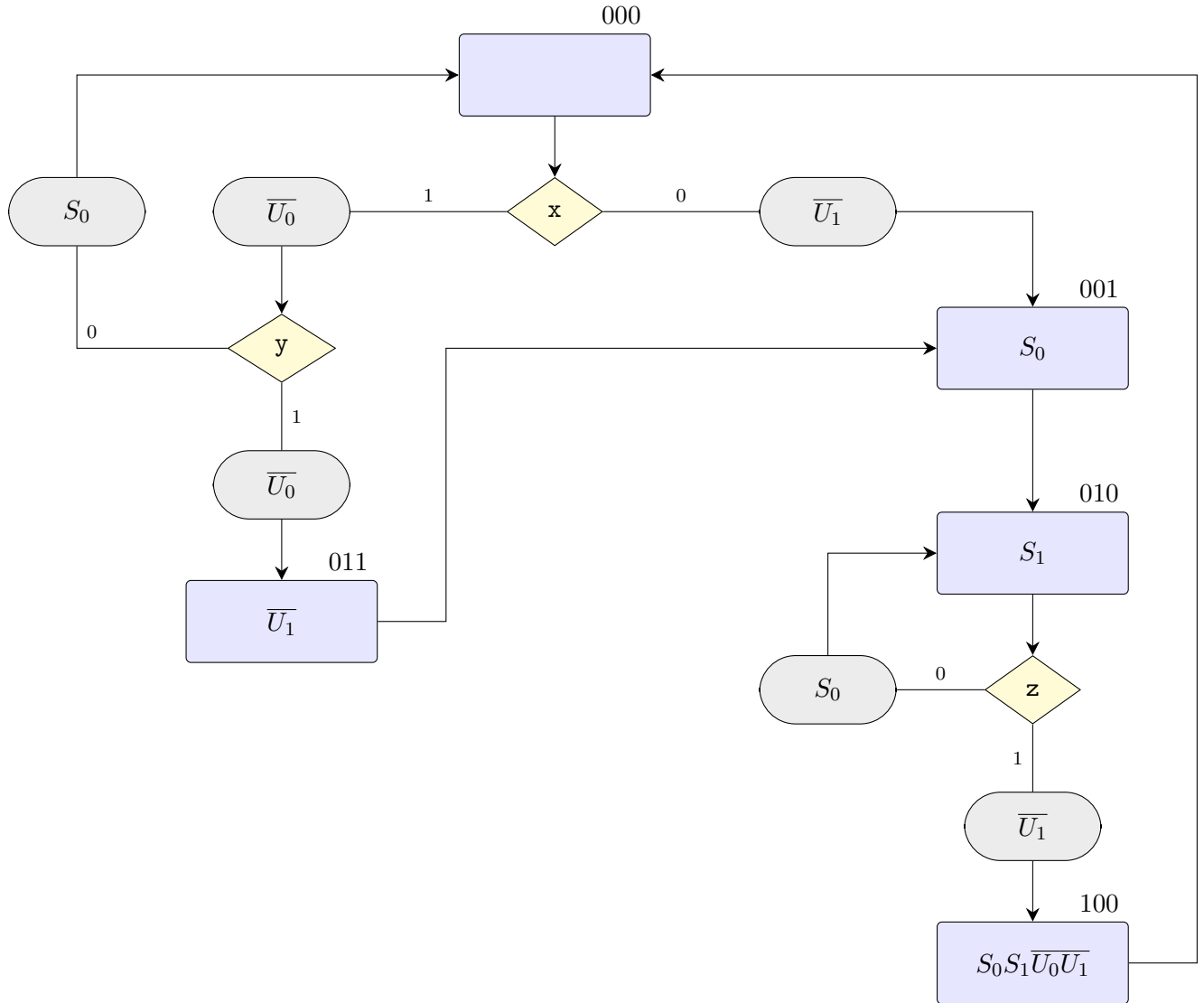


Figure 1: Carta ASM para la máquina de estados del punto 1.

Estado Actual			Entradas			Estado Siguiente			Salidas			
$P_2(t)$	$P_1(t)$	$P_0(t)$	X	Y	Z	$L_2(t+1)$	$L_1(t+1)$	$L_0(t+1)$	S_1	S_0	$\overline{U_1}$	$\overline{U_0}$
0	0	0	0	0	0	0	0	1	0	0	1	1
0	0	0	0	0	1	0	0	1	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0	1	1
0	0	0	0	1	1	0	0	1	0	0	1	1
0	0	0	1	0	0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0	0	0	0	1	0
0	0	0	1	1	0	0	1	1	0	0	1	0
0	0	0	1	1	1	0	1	1	0	0	1	0
0	0	1	0	0	0	0	1	0	0	1	1	1
0	0	1	0	0	1	0	1	0	0	1	1	1
0	0	1	0	1	0	0	1	0	0	1	1	1
0	0	1	0	1	1	0	1	0	0	1	1	1
0	0	1	1	0	0	0	1	0	0	1	1	1
0	0	1	1	0	1	0	1	0	0	1	1	1
0	0	1	1	1	0	0	1	0	0	1	1	1
0	0	1	1	1	1	0	1	0	0	1	1	1
0	1	0	0	0	0	0	1	0	1	1	1	1
0	1	0	0	0	1	1	0	0	1	0	0	1
0	1	0	0	1	0	0	1	0	1	1	1	1
0	1	0	0	1	1	1	0	0	1	0	0	1
0	1	0	1	0	0	0	1	0	1	1	1	1
0	1	0	1	0	1	1	0	0	1	0	0	1
0	1	0	1	1	0	0	1	0	1	1	1	1
0	1	0	1	1	1	1	0	0	1	0	0	1
0	1	1	0	0	0	0	0	1	0	0	0	1
0	1	1	0	0	1	0	0	1	0	0	0	1
0	1	1	0	1	0	0	0	1	0	0	0	1
0	1	1	0	1	1	0	0	1	0	0	0	1
0	1	1	1	0	0	0	0	1	0	0	0	1
0	1	1	1	0	1	0	0	1	0	0	0	1
0	1	1	1	1	0	0	0	1	0	0	0	1
0	1	1	1	1	0	0	0	1	0	0	0	1
0	1	1	1	1	1	0	0	1	0	0	0	1

Table 1: Tabla de transiciones y salidas de la máquina de estados.

Estado Actual			Entradas			Estado Siguiente			Salidas			
$P_2(t)$	$P_1(t)$	$P_0(t)$	X	Y	Z	$L_2(t+1)$	$L_1(t+1)$	$L_0(t+1)$	S_1	S_0	$\overline{U_1}$	$\overline{U_0}$
1	0	0	0	0	0	0	0	0	1	1	0	0
1	0	0	0	0	1	0	0	0	1	1	0	0
1	0	0	0	1	0	0	0	0	1	1	0	0
1	0	0	0	1	1	0	0	0	1	1	0	0
1	0	0	1	0	0	0	0	0	1	1	0	0
1	0	0	1	0	1	0	0	0	1	1	0	0
1	0	0	1	1	0	0	0	0	1	1	0	0
1	0	0	1	1	1	0	0	0	1	1	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0	0	0
1	0	1	0	1	0	0	0	0	0	0	0	0
1	0	1	0	1	1	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	0	0
1	0	1	1	0	1	0	0	0	0	0	0	0
1	0	1	1	1	0	0	0	0	0	0	0	0
1	0	1	1	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	1	0	0	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0	0	0
1	1	0	0	1	1	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	1	0	0	0	0	0	0	0
1	1	0	1	1	0	0	0	0	0	0	0	0
1	1	0	1	1	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0
1	1	1	0	1	0	0	0	0	0	0	0	0
1	1	1	0	1	1	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0	0	0
1	1	1	1	0	1	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	1	1	0	0	0	0	0	0	0

Table 2: Tabla de transiciones y salidas de la máquina de estados.

2. Encuentre el tamaño de la memoria necesaria y el contenido de memoria utilizando el direccionamiento por trayectoria.

Recuerde que antes de construir la tabla se debe asignar a cada estado de la carta ASM una representación binaria. Así mismo, recuerde que para cada estado es necesario considerar todas las posibles combinaciones de las variables de entrada, aun cuando algunas de ellas no se utilicen en determinado estado. Para esto se utiliza:

Tamaño de memoria estandar:

$$2^n \cdot (m)$$

En donde:

- n = Número de bits de estado + Número de entradas
- m = Número de bits liga + Número de salidas

En este caso tenemos 3 bits de estado y 3 bits de entrada, por lo que $n = 6$. Además, tenemos 3 bits de liga y 4 bits de salida, por lo que $m = 7$. Por lo tanto, el tamaño de memoria necesario es:

$$2^6 \cdot 7 = 448 \text{ bits}$$

Tamaño de memoria óptimo:

$$\text{Número de estados} \cdot 2^{\text{Número de estados}} \cdot (\log_2(\text{Número de estados})) + \text{Número de salidas}$$

$$5 \cdot 2^3 \cdot (\log_2(5)) + 4 = 96.8771238 \text{ bits}$$

3. Implementación de direccionamiento por trayectoria en Quartus.

La implementación se realizó en Quartus Prime 18.1 Lite, seleccionando la familia MAX 10 y el dispositivo 10M50DCF484C7G. El archivo practica_3.bdf se utilizó como entidad de nivel superior; en él se conectaron los bloques que forman la ruta de datos de la máquina: el divisor de frecuencia, el registro de estado, el concatenador de dirección, la memoria y el divisor de datos.

El funcionamiento es síncrono. El divisor de frecuencia genera el reloj lento a partir de la señal de la FPGA y el registro conserva el estado presente. Este estado se concatena con

las tres entradas externas para formar una dirección de seis bits. La memoria entrega una palabra de siete bits, formada por tres bits de liga y cuatro bits de salida. Finalmente, el divisor de datos separa ambos campos: la liga se conecta a la entrada del registro y las salidas se conectan a los LED de la tarjeta.

La dirección de memoria se construye mediante concatenación directa. La expresión `entradaB & entradaA` coloca primero el estado presente y después las entradas, por lo que cada combinación de estado y entradas selecciona una localidad distinta de la ROM.

```

1 entity concatenador_datos is
2     Port ( entradaA : in  STD_LOGIC_VECTOR(2 downto 0);
3           entradaB : in  STD_LOGIC_VECTOR(2 downto 0);
4           salida    : out STD_LOGIC_VECTOR(5 downto 0) );
5 end concatenador_datos;
6
7 architecture Behavioral of concatenador_datos is
8 begin
9     process (entradaA, entradaB)
10    begin
11        salida <= entradaB & entradaA;
12    end process;
13 end Behavioral;

```

Listing 1: Construcción de la dirección de memoria mediante el estado presente y las entradas.

El contenido de la carta ASM se almacenó en una ROM de 64 localidades de siete bits. Cada entrada de la tabla se escribe como la concatenación de la liga del estado siguiente con las cuatro salidas. Por ejemplo, las primeras ocho localidades corresponden al estado presente ‘000’ y contemplan las ocho combinaciones posibles de las tres entradas.

```

1 type rom_type is array (0 to 63) of std_logic_vector(6 downto 0);
2 signal internal_mem : rom_type := (
3     0 => "001" & "0011", -- Estado 000, entradas 000
4     1 => "001" & "0011", -- Estado 000, entradas 001
5     2 => "001" & "0011", -- Estado 000, entradas 010
6     3 => "001" & "0011", -- Estado 000, entradas 011
7     4 => "000" & "0011", -- Estado 000, entradas 100
8     5 => "000" & "0010", -- Estado 000, entradas 101
9     6 => "011" & "0010", -- Estado 000, entradas 110
10    7 => "011" & "0010", -- Estado 000, entradas 111
11    others => "000" & "0000"
12 );
13
14 data <= internal_mem(conv_integer(unsigned(dir)));

```

Listing 2: Declaración de la ROM y ejemplo de sus palabras de control.

La palabra obtenida de la memoria se divide en dos campos. Los tres bits más significativos representan la liga, es decir, el estado que será cargado en el siguiente flanco de reloj; los cuatro bits menos significativos representan las salidas actuales.

```
1 process (entrada)
2 begin
3     liga      <= entrada(6 downto 4);
4     salidas  <= entrada(3 downto 0);
5 end process;
```

Listing 3: Separación de la liga y las salidas de la palabra de memoria.

El registro de estado carga la liga únicamente en el flanco ascendente del reloj dividido. El reinicio es activo en bajo, por lo que al activar ‘RESET’ el estado se fuerza a ‘000’. Esta señalización coincide con las conexiones del diagrama esquemático y permite observar la secuencia de estados en la FPGA.

```
1 process (CLK, RESET)
2 begin
3     if RESET = '0' then
4         internal_value <= B"000";
5     elsif rising_edge (CLK) then
6         internal_value <= DATA_IN;
7     end if;
8 end process;
```

Listing 4: Registro del estado presente con reinicio activo en bajo.

Después de agregar los archivos VHDL y el archivo de diagrama al proyecto, se asignaron los pines de reloj, reinicio, entradas y salidas en el archivo de configuración de Quartus.

4. Demuestre el correcto funcionamiento de su carta ASM cargando su código por equipos a su FPGA y mostrándola a la profesora en el laboratorio.

5. Simule su diseño para probar su funcionamiento. Recuerden que en sus simulaciones debe aparecer el contenido de la memoria además del estado presente.

Análisis de Resultados

Discusión de Resultados

Conclusiones

Andrea Saldaña Navarrete - 320206384

Hansel Yael Tepal Briseño - 320078282

Luis Antonio Ugartechea González - 320223822

1 Bibliografía

- [1] D. K. Pérez Pérez, “Reporte Construcción de Máquinas de Estados Usando Memorias”, Laboratorio de Organización y Arquitectura de Computadoras, Facultad de Ingeniería, Universidad Nacional Autónoma de México, Semestre 2027-1, 2026.
- [2] Facultad de Ingeniería, “Capítulo III: Construcción de máquinas de estados usando memorias”, Apuntes de Arquitectura de Computadoras, Universidad Nacional Autónoma de México (UNAM). [En línea]. Disponible en: https://biorobotics.fi-p.unam.mx/wp-content/uploads/Courses/arquitectura_de_computadoras/2021-1/lecciones/Leccion_3_arquitectura_de_computadoras_2021_1.pdf
- [3] J. Savage, *Diseño de Microprocesadores*, Facultad de Ingeniería, Universidad Nacional Autónoma de México (UNAM). [En línea]. Disponible en: <http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf>